Dynamic programming (state tracking)

HW recap

$|s| = n$

```
for i in range(s.len)        -> xn
    print(s.substring(i, len))   -> O(n)
```

S: "hello"

hello

ello

llo

lo

o

total: $O(n^2)$

Recap

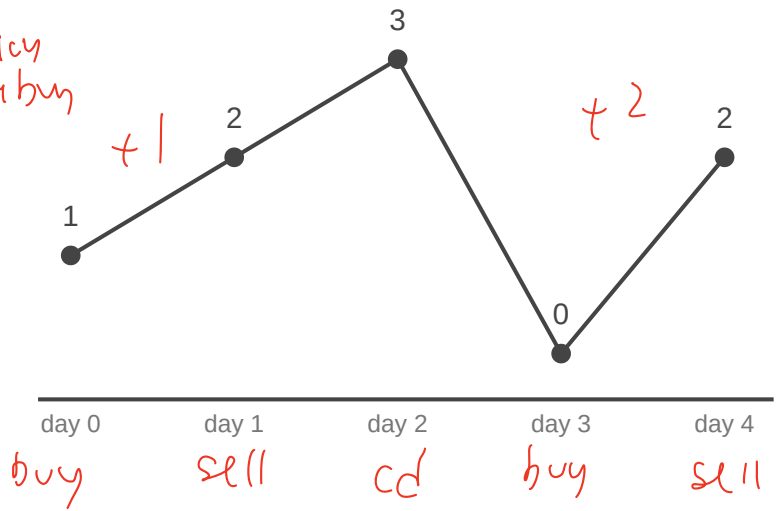
- dp[i] → house robber
- dp[i][j] → min path sum, LCS
- dp_state_1[i], dp_state_2[i], dp_state_3[i] ...

Buy and sell stock, with cooldown

1. pass
2. buy
3. sell → if sell, next day can't buy

1 / 1 stock

if no cd, max profit?



Buy and sell stock, with cooldown



have(1) = -1

no-stock(1) = 1

Problem statement

Input: `prices[i]` = the stock price on day i.

Rules:

- **Capacity 1.** Can keep 1 stock only.
- **Cooldown.** If **sell** on day i, cannot buy on day i+1.
- **Skip.** You may always pass and do nothing.

Output: the max total profit.

Step 1. Define subproblems - attempt 1

dp[i] = max profit by the end of day i

↳ dp[i] = dp[i-1] ?

↳ did I sell? cd.

Step 1. Define subproblems - attempt 2

Two states:

- `have[i]` = max profit by end of day i, **holding** a stock
- `no_stock[i]` = max profit by end of day i, **not holding**

Step 2. Recurrence - `no_stock`

```
no_stock[i] = max(
  no_stock[i-1],
  have[i-1] + prices[i])
```

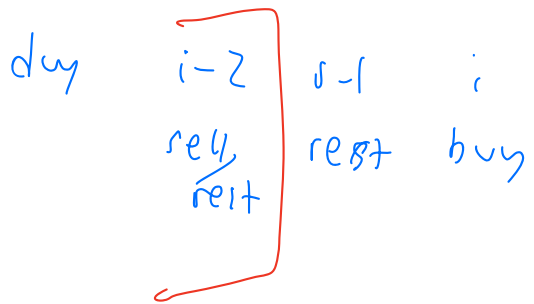
Step 2. Recurrence - have

```
have[i] = max(
  have[i-1],  → have stock yesterday, didn't sell
  no_stock[i-1] - prices[i]) # <- suspicious?
              ↳ we bought today
```

Step 2. Recurrence - both

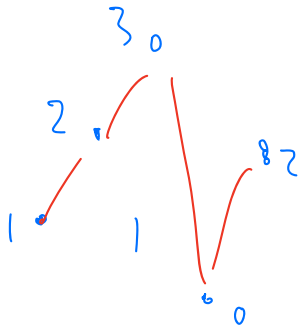
```
no_stock[i] = max(no_stock[i-1], have[i-1] + prices[i])
have[i]     = max(have[i-1], no_stock[i-2] - prices[i])
```

because cd rule!



The dp table

day	0	1	2	3	4
price	1	2	3	0	2
have	-1	-1	-1	1	1
no_stock	0	1	2	2	3



-1 + 2 = +1

-1 + 2 - 0 + 2 = 3

original one = max(have(n-1), no_stock(n-1))

= no_stock(n-1) (assume prices > 0)

The crux

1. **Define subproblems**
2. **Find recurrence**
3. **Specify order**
4. **Base case**

dp-state1(i)
dp-state2(i)

Code

```
have = [float('-inf')] * n
no_stock = [0] * n

have[0] = -prices[0]
no_stock[0] = 0

for i in range(1, n):
  free_to_buy = no_stock[i-2] if i >= 2 else 0

  have[i]     = max(have[i-1], free_to_buy - prices[i])
  no_stock[i] = max(no_stock[i-1], have[i-1] + prices[i])

return no_stock[n-1]
```

→ buy on first day

↳ iteration order ? does not matter.

can also eval no_stock first

Runtime & Space complexity

Runtime: O(n)

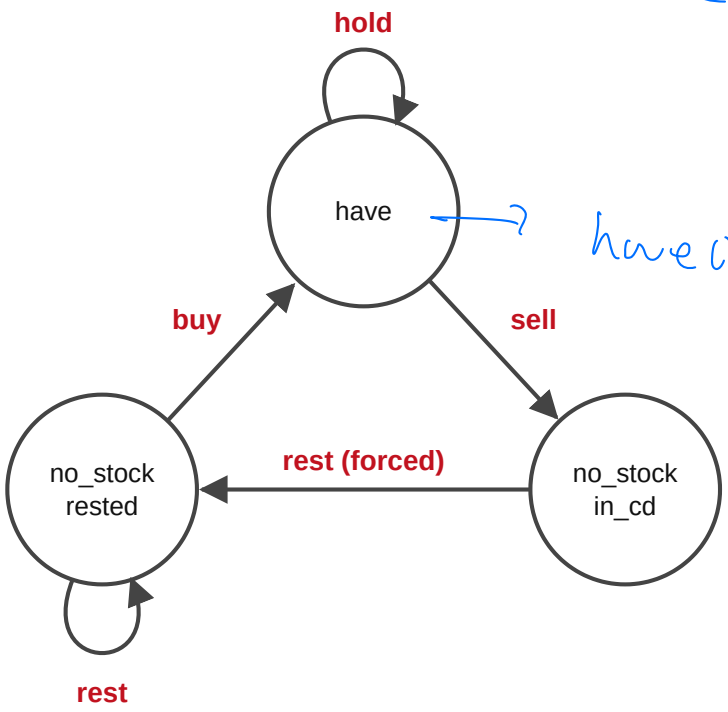
Space: O(n)



O(1)

hue(i-1)
no-stuck(i+1)
(i-2)

State machine, scaling to more states



market chain!

state of day i depends on...

state on day i-1 only

Step 2. Recurrence - alt

```
no_stock_in_cd[i]  = have[i-1] + prices[i]
no_stock_rested[i] = max(no_stock_rested[i-1],
  no_stock_in_cd[i-1])
have[i] = max(have[i-1], no_stock_rested[i-1] - prices[i])
```

i depends on i-1

Learn dp in 1 state

- multiple dp arrays, one for each state

Max subarray sum with one deletion

```
arr = [1, -2, 0, 3]
```

if no deletion → +3

if allow deletion → delete -2, (1,0,3) → 4

Problem statement

Input: an integer array `arr`.

Output: the largest sum of a **contiguous** subarray, where you may optionally delete **at most one** element.

Constraint: the result must be **non-empty** after the deletion.

dp[i] = max subarray sum for arr[0...i]
and .... ends exactly at e[i

What states do we need?

dp[i] = ? 1. max subarray sum for arr[0...i] w/o deleting anything and includes arr[i]

dp2[i] = ? 2. max subarray sum arr[0...i] but forced to delete exactly 1 and includes arr[i]

Step 1. Define subproblems

Both end **exactly at** index i:

- `keep[i]` = max subarray sum ending at i, **no deletion used**
- `del[i]` = max subarray sum ending at i, **deletion already used**

`arr[i]` is always kept.

Q: original problem's soln: max(keep[0...n-1], del[0...n-1])

↓

2 arrs!

The dp table

arr = [1, -2, 0, 3]

i	0	1	2	3
arr	1	-2	0	3
keep	1	-1	0	3
del	-inf	-2	1	4

del[0]: invalid. can't del & keep arr[0]

del[1] = -2, why? delet 1.

max sum, end exactly at i, no deletion.

Step 2. Recurrence

```
keep[i] = max ( arr[i], arr[i] + keep[i-1] )
del[i]  = max (
```

end at i, but must delete an el.

Cases

- delete arr[i-1]
  → keep[i-2] + arr[i] + arr[i]
- delete before i-1
  → de[i-1] + arr[i]

Step 2. Recurrence

```
keep[i] = max(keep[i-1] + arr[i], arr[i])
del[i]  = arr[i] + max(del[i-1], keep[i-2], 0)
```

max keep(i) keep(2), keep(3), ...?

keep(5) = ?

Code

```
keep = [NEG] * n
dele = [NEG] * n

# Base cases
keep[0] = arr[0]
dele[0] = NEG

for i in range(1, n):
  keep[i] = max(keep[i-1] + arr[i], arr[i])
  dele[i] = arr[i] + max(dele[i-1], 0)
  if i >= 2:
    dele[i] = max(dele[i], arr[i] + keep[i-2])

return max(max(keep), max(dele))
```

How is LCS useful irl?

- compare strings.

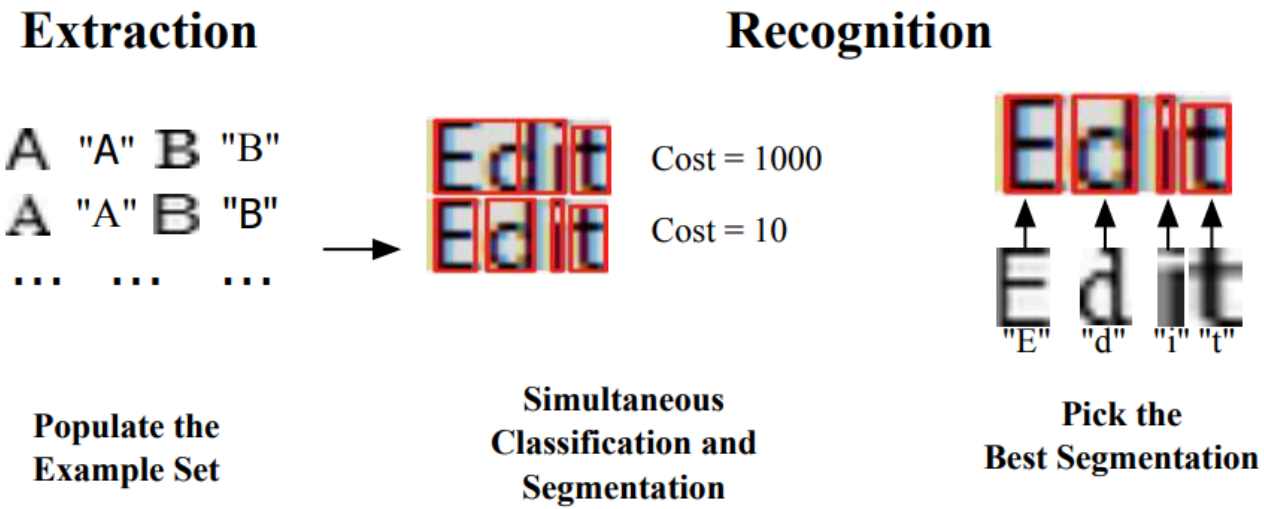
Recap: the whole DP arc

- **1D.** `dp[i]`. house robber, max subarray, word break
- **2D.** `dp[i][j]`. min path sum, LCS, edit distance, knapsack
- **States.** `dp[STATE][i,j,...]`. today. stocks, sum with deletion.

IRL

- My past
  - Undergrad research: screen OCR
  - AI startup: correlating AI-generated suggestions to text
- Other classes:
  - Applied math. Dynamic time warping
  - Comp bio. Comparing DNA sequences
  - AI. Viterbi algorithm. Audio waves -> most likely words.

Past research



optimal alignment of first i pixels of columns to c characters

Past research

$$OPT(c,i) = min_{1 \le j \le i} \{OPT(c-1,j) + bestMatch(j+1, i)\}$$

The structure of the problem lends itself nicely to a dynamic programming solution, where the subproblems entail finding the best way to segment the first few columns of *Img* into a certain number of characters. We are going to capture the result of this subproblem with the notation *OPT(c,i),* which represents the optimal cost of segmenting the first *i* columns of *Img* into *c* characters.

Exam material

- 1b
- mindmap 2D